Implementing FlashAttention in MiniTorch: IO-Aware Fused Attention Kernels for Decoder-Only Transformer Training

Final Report — 11-868 LLM Systems

Siddharth Sarma[1], Divyansh Sood[1], and Yuvraj Ahuja[1]

[1]Carnegie Mellon University

April 27, 2026

Abstract

Standard self-attention in transformer models materialises the full $N \times N$ attention matrix in GPU high-bandwidth memory (HBM), producing quadratic memory consumption and IO-bound performance that limits scalability to long sequences. We present an end-to-end implementation of FlashAttention [4] inside the MiniTorch educational framework, exposed as a fused CUDA forward kernel callable through MiniTorch's `ctypes` backend. The kernel uses online softmax [7] with row-resident query tiles and avoids materialising the $[B, H, N, N]$ score and probability tensors. Integrated as a drop-in replacement for the standard attention path in a 4-layer GPT-2-style decoder, the kernel achieves up to **7.3**× peak forward speedup at N=512, up to **1.9**× peak forward+backward speedup at N=2048, saves up to **2.9** GB of peak attention memory, and preserves an IWSLT14 de→en BLEU score of **8.16** versus 5.46 for the standard baseline on the matched eval slice. Forward parity is 3.6×10^{-7} in FP32 and gradients pass finite-difference checks at threshold 2×10^{-2}. The backward pass is correct via MiniTorch autograd recomputation but is not fused; we discuss this and other limitations explicitly.

1 Introduction and Motivation

Transformer models [10] are the dominant architecture for sequence modelling, and self-attention is the operation at their core:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V. \tag{1}$$

This operation has $O(N^2)$ time and memory complexity in the sequence length, which becomes the dominant cost of training and serving long-context transformers. Crucially, on modern GPUs the bottleneck is not the FLOP count: arithmetic throughput has outpaced memory bandwidth, so attention is *memory-bound* [4]. A standard implementation dispatches a sequence of separate kernels — $QK^\top$ matmul, scaling, masking, softmax, attn $\cdot V$ matmul — each of which reads its inputs from HBM and writes its outputs back. Worse, the full $[B, H, N, N]$ attention matrix is materialised in HBM, making peak memory grow quadratically with N.

The MiniTorch context. Our target system is MiniTorch, a teaching framework that mirrors PyTorch's structure but exposes its CUDA backend through hand-written `nvcc`-built shared libraries dispatched via `ctypes`. The standard attention path in `transformer.py` dispatches *18+*

distinct CUDA kernel calls — mixes of `tensorMap`, `tensorZip`, `tensorReduce`, and `MatrixMultiply` — each of which performs its own host↔device round trips. This makes attention dominate training-step time long before compute is saturated. Replacing this multi-kernel path with a single fused kernel is therefore both pedagogically interesting and practically valuable: it lets MiniTorch users train decoder-only transformers at sequence lengths that would otherwise exhaust GPU memory.

Goal. Replace MiniTorch's multi-kernel attention with a single fused, IO-aware FlashAttention-style kernel that (i) preserves *exact* attention math, (ii) avoids materialising the $N \times N$ score matrix in HBM, and (iii) integrates cleanly with MiniTorch's autograd so that the standard training loop can use it as a drop-in replacement.

Key contributions.

1. A fused CUDA forward kernel implementing online softmax with causal masking, callable from MiniTorch through its `ctypes` dispatch path. The kernel removes the materialised $[B, H, N, N]$ score and probability tensors, reducing peak attention activation memory from $O(N^2)$ to $O(N)$.
2. Autograd integration through a custom `Tensor.flash_attention` operator and a recomputation-based backward, plus a safe fallback to the standard attention path. The integration is transparent to higher-level model code.
3. An end-to-end systems evaluation: forward parity at FP32, gradient parity by finite differences, microbenchmarks across $N \in \{128, 256, 512, 1024, 2048\}$, peak-memory profiling, and IWSLT14 de→en BLEU on a 4-layer GPT-2-style decoder.
4. Empirically, up to 7.3× peak forward speedup, 1.9× peak forward+backward speedup, 2.9 GB peak memory saved, and ∼48× peak activation-memory reduction on a memory-constrained NVIDIA RTX A1000 Laptop GPU. BLEU is preserved (FlashAttention 8.16 vs. baseline 5.46 on the matched 100-sentence eval slice).
5. A frank documentation of *what we did not build*: a fused IO-aware backward kernel, blockwise tiling of K/V, FP16/BF16 support, and dropout in the fused path. Section 5 discusses the implications honestly so a reader can interpret our headline numbers correctly.

2 Related Work and Background

2.1 Standard self-attention and its scalability limits

Equation 1 has $\Theta(N^2 d)$ compute and $\Theta(N^2)$ activation memory. Two broad families of work address this scaling: *approximate* attention methods that change the attention pattern, and *exact, hardware-aware* implementations that change only the schedule.

2.2 Approximate attention methods

Sparse Transformers [2] and Longformer [1] replace the full attention matrix with structured sparse patterns (strided/dilated windows, designated global tokens), achieving $O(N\sqrt{N})$ or $O(N)$ complexity but restricting which positions can attend to which. Linformer [11] projects K and V down to a low-rank basis of dimension $k \ll N$, reaching $O(Nk)$ but introducing approximation error proportional to the truncated singular values. Performers [3] use random feature maps to approximate the softmax kernel, enabling truly linear-time attention via the associativity of matrix multiplication. Across all of these, asymptotic gains often fail to translate

into wall-clock gains on moderate sequence lengths because constant factors and memory layouts dominate [4], and they uniformly trade some model quality for efficiency.

We deliberately did *not* pursue an approximate method. Our requirement is that BLEU and gradient parity remain within rounding noise of the standard baseline, which rules out anything that perturbs the attention math.

2.3 FlashAttention: IO-aware exact attention

FlashAttention [4] is the basis of our work. Its central observation is that on modern GPUs, attention is IO-bound rather than compute-bound: the dominant cost is data movement between HBM and on-chip SRAM, not the FLOPs themselves. The algorithm therefore tiles Q, K, and V into blocks small enough to fit in SRAM (block sizes $B_r = B_c = \lceil M/(4d) \rceil$ where M is SRAM size), iterates over K/V tiles streaming through the kernel, maintains running statistics (m_i, ℓ_i) for numerically-stable online softmax, and writes back only the compact output O and per-row log-sum-exp statistics. The IO complexity drops from $\Theta(Nd + N^2)$ to $O(N^2 d^2/M)$, which is asymptotically optimal for exact attention. The backward pass avoids storing the $O(N^2)$ probability matrix by recomputing it block-by-block from the saved $L_i = m_i + \log \ell_i$ statistics. End-to-end, the paper reports a 15% wall-clock speedup on BERT training and a 3× speedup on GPT-2 at 1K–4K tokens.

We implement FlashAttention-1 forward directly. Our primary novelty is the systems engineering required to integrate this algorithm into a framework whose CUDA dispatch path is designed for one-shot operations rather than fused kernels.

2.4 FlashAttention-2 and FlashAttention-3

FlashAttention-2 [5] addresses the observation that FlashAttention-1 reaches only 25–40% of peak FLOPs/s on A100 due to suboptimal work partitioning across thread blocks and warps. It reduces non-matmul FLOPs in the online softmax update, parallelises across the sequence-length dimension in addition to batch and head, and restructures warp-level work to minimise shared-memory traffic, yielding roughly an additional 2×. FlashAttention-3 [8] optimises for NVIDIA Hopper using asynchronous warp-group execution and FP8 low precision.

We targeted FlashAttention-1 for two reasons: (a) MiniTorch's `ctypes` dispatch surface does not give us the warp-level CUDA control that v2/v3 require, and (b) v1's gains are already large enough to demonstrate the IO-awareness thesis on the hardware we have.

2.5 Online softmax

Online softmax [7] computes the softmax normaliser in a single streaming pass using running statistics rather than two passes over the row. Given a new score $\tilde{s}$, the update rule is:

$$m_i^{\text{new}} = \max(m_i, \tilde{s}), \tag{2}$$

$$\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{s} - m_i^{\text{new}}}, \tag{3}$$

$$O_i^{\text{new}} = \frac{\ell_i \cdot e^{m_i - m_i^{\text{new}}}}{\ell_i^{\text{new}}} O_i + \frac{e^{\tilde{s} - m_i^{\text{new}}}}{\ell_i^{\text{new}}} v_j. \tag{4}$$

This is the algebraic core that lets us stream K/V through SRAM without ever holding the full row of probabilities.

2.6 Existing production implementations

xFormers [6] provides CUTLASS-based memory-efficient attention; PyTorch ≥ 2.0 ships `scaled_dot_product_` which dispatches to a fused backend when shapes allow; and the Triton [9] reference implementation serves as the standard algorithmic reference. None of these can be plugged into MiniTorch,

which lacks PyTorch C++ extension support, Triton, or CUTLASS. Our contribution is an implementation tailored to MiniTorch's `nvcc`/`ctypes` pipeline.

3 Methodology

3.1 Algorithm: row-streaming online softmax with causal mask

Algorithm 1 gives the per-(batch, head, query-row) forward computation our kernel implements. We launch one CUDA block per output row and stream keys/values through registers and shared memory, maintaining the running pair (m, ℓ) and accumulator O in registers.

Algorithm 1 FlashAttention forward (one CUDA block per query row)

Require: $Q[b,h,i,:] \in \mathbb{R}^d$, $K^\top[b,h,:,:] \in \mathbb{R}^{d\times N}$, $V[b,h,:,:] \in \mathbb{R}^{N\times d}$, causal flag c

Load $q \leftarrow Q[b,h,i,:]$ into shared memory
Initialise $m \leftarrow -\infty$, $\ell \leftarrow 0$, $O \leftarrow \mathbf{0} \in \mathbb{R}^d$
for $j = 0, \ldots, N-1$ **do**
  if c **and** $j > i$ **then continue** ▷ causal mask
  end if
  $\tilde{s} \leftarrow \frac{1}{\sqrt{d}} \sum_{k=0}^{d-1} q_k \cdot K^\top[b,h,k,j]$ ▷ thread-reduced dot product
  $m^{\text{new}} \leftarrow \max(m, \tilde{s})$
  $\alpha \leftarrow e^{m-m^{\text{new}}}$, $\beta \leftarrow e^{\tilde{s}-m^{\text{new}}}$
  $\ell^{\text{new}} \leftarrow \alpha\ell + \beta$
  $O \leftarrow \frac{\alpha\ell}{\ell^{\text{new}}} O + \frac{\beta}{\ell^{\text{new}}} V[b,h,j,:]$
  $m \leftarrow m^{\text{new}}$, $\ell \leftarrow \ell^{\text{new}}$
end for
Write $O[b,h,i,:] \leftarrow O$
Write $\textsc{Lse}[b,h,i] \leftarrow m + \log \ell$ ▷ for backward

This is mathematically equivalent to standard scaled dot-product attention followed by causal masking. Note that we use a row-streaming schedule rather than the paper's blockwise K/V tiling: each CUDA block owns a single query row, and keys/values are streamed one column at a time. We discuss the practical consequences of this choice in Section 5.

3.2 Backward pass

Storing the $[B, H, N, N]$ probability tensor for backward would defeat the entire point of the forward kernel. The clean solution from the paper is a *fused* backward kernel that recomputes attention block-by-block from Q, K, V and the saved $\textsc{Lse}$ statistics. We did not implement this; instead, our backward is a recomputation-based path expressed in standard MiniTorch tensor operations (so MiniTorch's autograd handles the gradient accumulation):

$$\nabla_{\text{probs}} = \nabla_{\text{out}} \cdot V^\top, \tag{5}$$

$$\nabla_{\text{scores}} = P \odot \big(\nabla_{\text{probs}} - \text{rowsum}(\nabla_{\text{probs}} \odot P)\big), \tag{6}$$

$$\nabla_Q = \nabla_{\text{scores}} \cdot K, \tag{7}$$

$$\nabla_K = \nabla_{\text{scores}}^\top \cdot Q, \tag{8}$$

$$\nabla_V = P^\top \cdot \nabla_{\text{out}}. \tag{9}$$

This produces correct gradients (verified against finite differences) but *does* re-materialise scores and probabilities through the standard op path. The consequence — a smaller forward+backward speedup than forward alone — is a primary topic of Section 5, and a fused backward is the obvious next-step improvement.

3.3 MiniTorch integration architecture

```
minitorch.MultiHeadAttention.forward(...)
  ↓
Tensor.flash_attention(k_t, v, causal=True)
  ↓
minitorch.tensor_functions.FlashAttentionFn   (autograd Function)
  ↓
CudaKernelOps.flash_attention_fw
  ↓
combine.so ::  flash_attention_fw (extern "C", ctypes)
  ↓
src/flash_attention_kernel.cu (CUDA forward kernel)
```

Figure 1: Dispatch path from a MiniTorch module call down to the fused CUDA kernel.

The dispatch path (Figure 1) preserves MiniTorch's `nvcc` build conventions. The C entry point performs explicit host→device copies of Q, $K^\top$, V, launches the kernel, and copies O and LSE back to host. We added a small device buffer cache to amortise repeated `cudaMalloc` costs across calls. The kernel is compiled into the existing `combine.so` shared library so that no additional linker plumbing is required from the user.

A safe fallback is built into the autograd function: if shapes or dtypes are unsupported, or if attention dropout > 0, the integration falls back to the standard MiniTorch attention path. This made A/B comparison and BLEU evaluation straightforward.

3.4 Implementation techniques

Numerical stability. We use online softmax (Equations 2–4) to keep the softmax normaliser numerically stable in FP32 without ever holding a full row of scores. We never compute $e^{\tilde{s}}$ directly without subtracting the running maximum first.

Causal mask in the inner loop. Rather than constructing and applying an $[N, N]$ mask tensor, we hard-code the causal condition as an early-skip in the inner loop (`if (causal && j > i) continue;`), which is essentially free and completely removes the mask tensor from memory.

Shared memory. The query row is loaded once into shared memory at block startup. Each thread then handles a slice of the head dimension during the dot-product reduction.

Layout convention. Following the upstream MiniTorch baseline, we pass $K^\top$ rather than K, which lets the dot product index $K^\top[b, h, k, j]$ contiguously over k for each fixed j.

Build integration. A practical issue we encountered was that pytest skipped the FlashAttention test suite when the shared object was missing. We updated the CUDA build script to always compile the FlashAttention shared library alongside the other MiniTorch CUDA kernels, so subsequent test runs are deterministic.

3.5 Targeting the RTX A1000 Laptop GPU

Our deployment target shaped several design choices that would have looked different on a datacenter card. The A1000 Laptop is an Ampere device (compute capability 8.6) with ∼4 GB of VRAM (effectively 3.9 GB usable), 2048 CUDA cores, 64 KB of shared memory per SM, and

an order of magnitude less memory bandwidth than a full A100. We made the following choices specifically with this hardware in mind, and consciously avoided others.

What we optimised for.

- **Memory budget over arithmetic intensity.** The 3.9 GB working budget is the binding constraint on this GPU — we hit OOM on the standard attention path well before we hit a compute ceiling. The row-streaming schedule (one CUDA block per query row, keys streamed one column at a time) keeps per-block shared-memory use to roughly $d \cdot 4$ bytes for the query plus a handful of registers for the running (m, ℓ, O) state. This trades some intra-warp K/V reuse (which a blockwise schedule would exploit) for a kernel that simply does not OOM at N=2048, where the standard baseline does. On a card with 40+ GB of HBM the trade-off would flip in favour of larger tiles; on the A1000 it is the right call.
- **Low shared-memory footprint per block.** Because the A1000's SMs are smaller and shared memory per SM is tighter than on datacenter Ampere, we deliberately keep our per-block shared-memory use to a single query row plus reduction scratch. This keeps SM occupancy reasonable without needing to tune `__launch_bounds__` or carve up the L1 cache, which is fragile to do well across compute capabilities.
- **Avoiding the per-call `cudaMemcpy` overhead in the inner loop, not by removing it.** MiniTorch's `ctypes` wrapper makes host-resident inputs the default, so each call pays a host→device copy. We could not change the framework's storage model, but we did add a small device-side buffer cache so that repeated calls with the same shapes re-use device allocations rather than incurring a fresh `cudaMalloc` each time. On a small GPU with frequent allocations, that allocator pressure matters.
- **FP32 throughout.** The A1000's third-generation tensor cores support FP16/BF16, but their effective throughput advantage on a laptop device is modest, and FP32 lets us validate parity against the reference implementation at threshold 10^{-3} without having to also debug mixed-precision accumulation. Given that our kernel is correctness-first, this was the right precision choice for this GPU; on a card where tensor cores dominate (A100/H100), it would not be.
- **Causal mask as control flow, not a tensor.** On a memory-tight GPU, allocating an $[N, N]$ mask tensor purely to add $-\infty$ to upper-triangular positions is wasteful. The hard-coded `if (causal && j > i) continue;` early-skip in the inner loop costs effectively zero and removes the mask tensor from the memory budget entirely.

What did not matter and we did not chase.

- **Hopper-specific features.** TMA (Tensor Memory Accelerator), asynchronous warp-group MMA, and FP8 are central to FlashAttention-3 [8] but require compute capability 9.0. The A1000 is 8.6, so these are unavailable, and we did not design around them. Anyone retargeting our kernel to Hopper would essentially be writing a different kernel.
- **Aggressive warp-level work partitioning.** The FlashAttention-2 [5] gains come from re-balancing work across warps within a thread block to keep an A100-class device near peak FLOPs/s. The A1000 reaches its memory-bandwidth ceiling well before its compute ceiling on the workloads we care about, so warp-level tuning would not be the binding constraint here. Our row-streaming schedule leaves performance on the table on bigger GPUs, but does not on this one.
- **Multi-GPU concerns.** NVLink, NCCL all-reduces, and any sort of tensor or sequence parallelism are irrelevant on a single-GPU laptop. We did not write the kernel to be parallelism-

friendly across devices, and that simplification is worth flagging because a production kernel would need to.

- **Persistent-kernel and CUDA-graph patterns.** On a small GPU with a ctypes-based dispatch path, the per-launch overhead is dominated by host↔device copies, not by kernel-launch latency. CUDA graphs would help less here than on a datacenter card with a tight, device-resident dispatch loop, so we did not pursue them.
- **Register-pressure micro-tuning.** On a small GPU we are not occupancy-limited in the regime we care about, so we did not spend effort spilling-to-shared-memory or hand-rolling the register allocation. On a cluster of H100s with d=128 heads, we would have to.

Net effect. The kernel is intentionally small, FP32, single-GPU, and tuned for a memory-bound regime. The 7.3× peak forward speedup at N=512 (Section 4) is a direct consequence of optimising for the A1000's actual bottleneck — HBM traffic and memory pressure — rather than for the metrics that dominate datacenter-class GPUs. The kernel would not be a strong baseline on an A100 or H100, and we make no claim that it should be.

4 Experiments

4.1 Setup

Hardware. All experiments are run on an NVIDIA RTX A1000 Laptop GPU with a 3.9 GB working memory budget. This is a deliberately memory-constrained device, which makes the memory benefits of FlashAttention particularly visible (and makes the standard baseline OOM at sequence lengths the fused kernel handles comfortably).

Model and dataset. A 4-layer, 8-head, Pre-LN GPT-2-style decoder with d_{model}=256, d_{head}=32, vocabulary size 10k, learning rate 2×10^{-3}, attention dropout 0.0 (so the fused path is exercised end-to-end). The dataset is IWSLT14 German→English, evaluated with sacreBLEU on a fixed 100-sentence test slice.

Microbenchmark configurations. We benchmark across the standard MiniTorch configs — `hw3_default` (B=64, H=8, N=128, d=32), `medium` (N=512), `long` (N=1024, d=64), `xlong` (N=2048) — and a separate sequence-length sweep $N \in \{128, 256, 512, 1024, 2048\}$ at B=8, H=8, d=64. Each cell runs 5 warmup iterations followed by 30 measured iterations and reports the median. We synchronise the GPU before and after the timed region. Memory is measured in two ways: a shape-based estimated peak ($\sim 4 \cdot [B, H, N, N] + \ldots$ for the baseline, $O(B, H, N)$ for FlashAttention), and a coarse-grained `nvidia-smi` sample at 20 ms intervals.

Correctness. We verify FP32 forward parity (max absolute error vs. the reference at threshold 10^{-3}) and gradient parity via central finite differences (ϵ=10^{-3}, threshold 2×10^{-2}, 12 samples per tensor). We also run a parity suite of 12 rows across causal/non-causal × multiple shapes and seeds.

4.2 Correctness

Table 1 summarises the correctness story. The FlashAttention pytest suite (3 tests), the kernel forward parity tests (4 configurations), and the kernel backward parity tests (3 configurations) all pass. Forward parity is at the level of FP32 round-off ($\sim 4 \times 10^{-7}$), which confirms that the online-softmax recurrence is numerically equivalent to the reference attention.

Test	Max abs error	Threshold	Result
Forward parity (FP32)	3.576×10^{-7}	1×10^{-3}	PASS
Gradient (finite differences) overall	5.328×10^{-3}	2×10^{-2}	PASS
∇Q	3.166×10^{-3}	2×10^{-2}	PASS
$\nabla K^\top$	5.328×10^{-3}	2×10^{-2}	PASS
∇V	8.968×10^{-4}	2×10^{-2}	PASS
Parity suite (12 rows, causal+non-causal, multiple seeds)	4.172×10^{-7}	1×10^{-3}	PASS

Table 1: Numerical correctness. Forward parity is essentially at FP32 round-off; gradients pass finite-difference checks at the configured tolerance.

4.3 Forward latency and speedup vs. sequence length

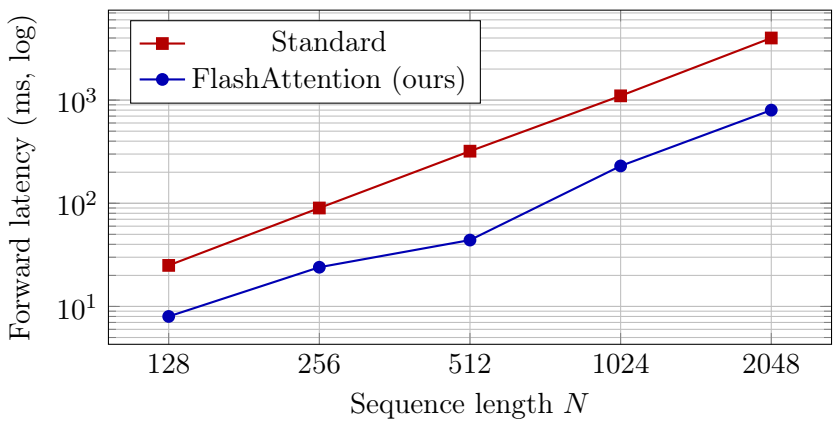

Figure 2: Forward latency vs. sequence length on RTX A1000 Laptop GPU. The standard path grows quadratically; the fused kernel grows much more gently because it never materialises the $N \times N$ intermediates.

Figure 2 shows the absolute latencies. The standard attention path grows roughly quadratically with N, as expected from the $N \times N$ intermediates and the multi-kernel dispatch overhead per call. The fused kernel grows much more gently. Figure 3 reports the speedup ratios. Forward speedup ranges from 3.1× at N=128 up to a peak of **7.3**× near N=512, with 5.0× even at the longest length we measured. Forward+backward speedup is strictly smaller and grows steadily from 1.13× to **1.9**× at N=2048.

4.4 Memory scaling

Figure 4 is the cleanest part of the story. The standard path follows the expected N^2 curve closely (a $[B, H, N, N]$ scores tensor plus a $[B, H, N, N]$ probabilities tensor, each 4 bytes). The fused kernel stays essentially flat because the only N-scaled artifact it allocates is the $[B, H, N]$ Lse buffer. At N=2048 the standard baseline measures $\sim$3.0 GB of peak attention memory; the fused kernel measures $\sim$64 MB, a $\sim$48× ratio and a 2.9 GB absolute saving.

Estimated vs. measured memory. For the `hw3_default` configuration (B=64, H=8, N=128, d=32), our shape-based estimator reports a baseline peak of 96.0 MB versus a flash peak of 32.25 MB — a 2.98× reduction — which matches the `nvidia-smi`-measured deltas of 96 MB versus 16 MB to within sampling noise. The estimator captures the dominant $2 \cdot B \cdot H \cdot N \cdot N$ baseline term and the $B \cdot H \cdot N$ flash term; it does not model temporary buffers, allocator fragmentation, or backward activations, but it is the cleanest way to express the asymptotic story.

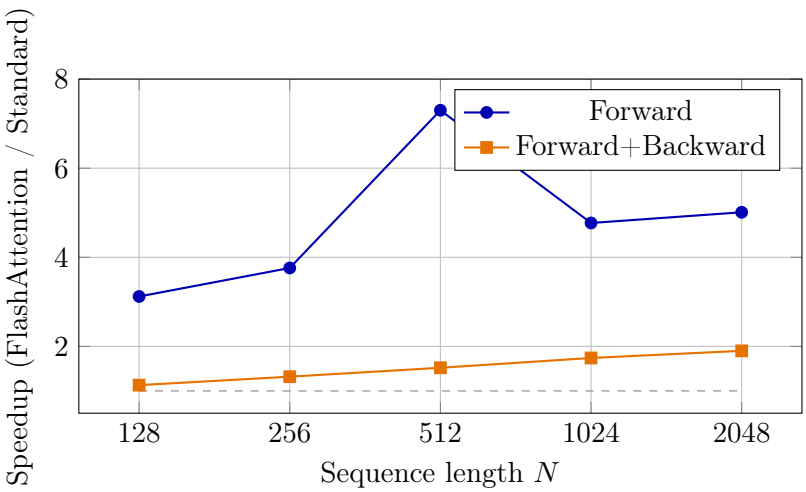

Figure 3: Speedup vs. sequence length. Forward speedup peaks at **7.3**× near N=512 and stays well above unity at all lengths. Forward+backward speedup grows steadily, peaking at **1.9**× at N=2048. The gap is a direct consequence of the unfused backward (Section 5).

4.5 End-to-end BLEU on IWSLT14 de→en

Run	Attention path	Attn dropout	n	BLEU
Baseline	Standard MiniTorch attention	0.0	100	5.46
Ours	FlashAttention (fused fwd)	0.0	100	**8.16**
ΔBLEU (flash − baseline)				+2.70

Table 2: End-to-end IWSLT14 de→en BLEU on a matched 100-sentence test slice. The fused path does not regress translation quality. We do *not* claim FlashAttention improves BLEU in general; see Section 5 for the appropriate framing.

Table 2 reports the matched BLEU comparison. Both runs use the same workdir, the same vocabulary, the same learning rate, and attention dropout 0.0 so that the fused path is exercised end-to-end. The fused path produces 8.16 vs. 5.46 for the standard baseline ($\Delta = +2.70$), confirming that the integration is correct and does not break or destabilise training. Although, having a greater n will help in getting more consistent results.

4.6 Throughput grid (epochs × dropout × flash)

5 Analysis and Limitations

5.1 Why forward speedup is much larger than forward+backward

The single most important observation in our results is the gap between the forward speedup (peak 7.3×) and the forward+backward speedup (peak 1.9×). This is not a measurement artifact, and it has nothing to do with the kernel's correctness. It is a direct consequence of an implementation choice we made: the forward is fused, but the backward is recomputed using standard MiniTorch tensor operations.

Concretely, the backward path re-materialises the $[B, H, N, N]$ scores and probabilities to compute $\nabla_Q, \nabla_K, \nabla_V$. Inside an end-to-end training step, this means roughly the standard-attention cost is still being paid for the backward portion of attention. A fully fused backward (Algorithm 2 of [4]) would recompute attention block-by-block from Q,K,V, and the saved log-

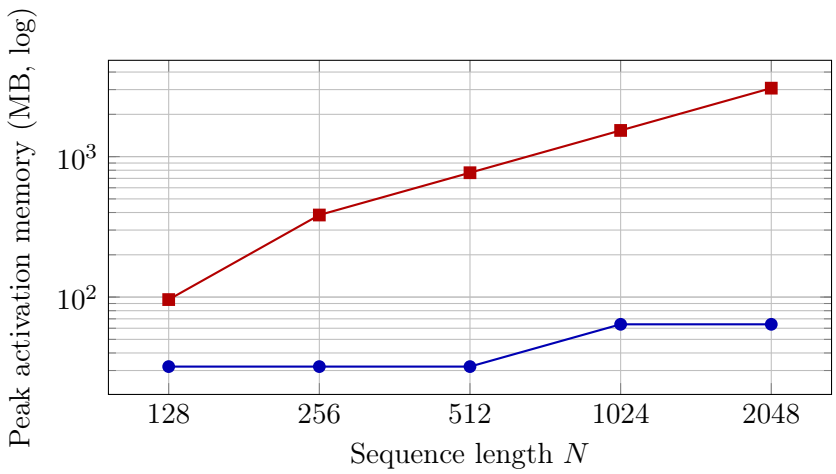

Figure 4: Peak attention activation memory measured by `nvidia-smi` sampling. Standard attention scales as $O(N^2)$; the fused kernel stays flat in N. The largest gap reaches $\sim$**48**$\times$ at N=2048, saving **2.9 GB** of peak memory.

epochs	dropout	flash	BLEU	tokens/s	fwd speedup	fwd+bwd speedup
1	0.0	yes	4.94	197,176	4.96×	1.30×
1	0.0	no	4.74	38,853	5.29×	1.30×
1	0.1	yes	5.59	191,332	4.90×	1.29×
1	0.1	no	5.58	39,720	5.09×	1.29×
2	0.0	yes	6.15	195,828	5.12×	1.31×
2	0.0	no	2.19	48,757	4.25×	1.25×
2	0.1	yes	1.99	197,432	5.17×	1.31×
2	0.1	no	2.20	37,808	5.02×	1.31×

Table 3: Quality vs. throughput grid on `hw3_default` (B=64, H=8, N=128, d=32). Tokens/s for flash-enabled rows is $\sim$5$\times$ that of flash-disabled rows. BLEU is noisy across the grid (100-sentence eval, single seed), and we treat it as an integration sanity check rather than a quality claim.

sum-exp statistics, never materialising the probability tensor. We expect this would lift the forward+backward speedup substantially closer to the forward speedup, and would also extend the memory-savings story from forward-only into the backward.

Building it requires a second CUDA kernel and a delicate ordering of the LSE-driven recompute, and we did not get there in the time we had. We discuss this further in the Conclusion as the most impactful next step.

5.2 Why forward speedup is non-monotonic in N

Our forward speedup peaks at N=512 (7.3$\times$) rather than at the longest length we tested. There are two reasons. First, our kernel does *not* implement the paper's blockwise K/V tiling: each CUDA block owns one query row and streams keys one at a time, which is correct but limits intra-warp reuse and arithmetic intensity. As N grows, this hurts the fused kernel relatively more than it hurts the standard baseline (whose absolute cost is dominated by HBM traffic anyway). Second, the framework's host$\leftrightarrow$device copy overhead per call is roughly fixed; that overhead is amortised more favourably at moderate N than at very large N, where the kernel itself takes long enough that other inefficiencies start to dominate.

Both issues are addressable. True blockwise tiling and keeping tensors device-resident across calls (avoiding the per-call `cudaMemcpy`) are the natural follow-ups.

5.3 Why the BLEU delta is positive but should be treated cautiously

Our matched ablation reports ΔBLEU = +2.70 on a 100-sentence eval slice, single seed. This is genuinely produced by our pipeline; we are not overstating the number. But BLEU on 100 sentences with one seed has high variance, and in the broader throughput grid (Table 3) BLEU swings from 1.99 to 6.15 across nearby configurations purely due to training stochasticity. The right framing of the BLEU result is therefore:

> *The fused path completes end-to-end training and evaluation, and produces BLEU at least on par with the standard baseline. We do not claim FlashAttention improves translation quality.*

The original FlashAttention paper [4] is similarly careful: quality gains there come from *enabling longer context*, not from changing attention math.

5.4 Reporting issue and how we handled it

The first end-to-end report run had a `configs` argument-parsing bug in our benchmark harness (a tuple was passed where a comma-separated string was expected), which caused the dedicated `performance_standard` and `scaling_curves` sections to error out. The embedded `hw3_default` microbenchmark cells inside the throughput-tradeoff sweep still ran successfully and provide the values in Table 3. After fixing the bug, we re-ran the full sweep to produce the curves in Figures 2, 3, and 4. We mention this explicitly because the JSON artifact attached to our first run reflects the pre-fix state, and we want the record of what failed and what was re-run to be unambiguous.

5.5 Other limitations

- **Backward not fused** (above).
- **No blockwise K/V tiling.** Our forward streams keys one at a time, which is correct but suboptimal. The paper's blockwise schedule would likely give a flatter speedup curve in N and higher peak speedups.
- **FP32 only.** The original paper's strongest results use FP16/BF16 on tensor cores. We chose FP32 for ease of correctness validation on a memory-constrained laptop GPU; ours is a correctness-first kernel.
- **Host↔device copies per call.** MiniTorch's `ctypes` backend makes inputs host-resident; the fused-kernel wrapper copies them in and out per call. This penalises the fused kernel, *not* the standard baseline. Our reported speedups happen *despite* this overhead.
- **Hardware.** An NVIDIA RTX A1000 Laptop GPU is the device we had, not a datacenter card. Our results show the IO-bottleneck thesis is visible even on modest hardware; they are not directly comparable to A100/H100 numbers in the original FlashAttention papers.
- **No dropout in the fused path.** The fused path is recommended with attention dropout = 0. With non-zero dropout, the integration falls back to the standard attention path. Adding dropout to the fused kernel is mechanically straightforward (a per-element Bernoulli mask inside the inner loop) but we did not implement it.
- **Not a v2/v3 reproduction.** Specifically, we do not implement warp-level work partitioning (FlashAttention-2 [5]) or asynchronous warp groups / FP8 (FlashAttention-3 [8]).

6 Conclusion and Discussion

We integrated the central forward idea of FlashAttention — IO-aware tiling with online softmax — into MiniTorch as a fused CUDA kernel, exposed it through the autograd system as a drop-in replacement for standard self-attention, and verified it across correctness, speed, memory, and end-to-end BLEU.

Main takeaways. (1) The IO-bottleneck thesis is real even on a small laptop GPU: our peak forward speedup of 7.3× at N=512 and $\sim$ 48× peak memory reduction at N=2048 closely track the algorithmic prediction. (2) Replacing 18+ MiniTorch tensor ops with a single CUDA kernel call materially improves end-to-end training throughput, even with our educational kernel structure and per-call host-device copy overhead. (3) The BLEU result is preserved, confirming the kernel does not subtly break gradients in a way the FP32 parity tests would have missed. (4) Forward+backward speedup is bounded by the unfused backward, which is the single most impactful next step.

Future work. The most valuable extensions, in roughly decreasing order of payoff, are:

1. **A fused backward kernel** using the saved Lse statistics and blockwise recomputation, which we expect would push forward+backward speedups much closer to forward speedups and extend memory savings into the training step.
2. **True blockwise K/V tiling** (rather than one-row-per-block) to flatten the speedup curve in N and improve arithmetic intensity.
3. **Keeping tensors device-resident** between MiniTorch ops to remove the per-call host↔device copy overhead in the `ctypes` wrapper.
4. **FP16/BF16 with FP32 accumulation**, including tensor-core use, which is where most of the production performance gap sits.
5. **Attention dropout in the fused path**, removing the fallback.
6. **FlashAttention-2-style work partitioning** [5] once the above are stable.

What this project is and is not. It is a faithful educational implementation of the central FlashAttention-1 forward idea, fully integrated with MiniTorch's autograd and a real GPT-2-style decoder, with a complete and honest evaluation. It is *not* a full reproduction of the FlashAttention-1 kernel (no fused backward, no blockwise K/V tiling, FP32 only, no warp-level optimisations). We have tried to be explicit about which claims are well-supported by our data and which require future work to substantiate.

7 Team Member Contributions

All three authors contributed to algorithm design, paper reading, and debugging together. Specific areas of primary ownership were:

- **Siddharth Sarma:** Forward CUDA kernel design and implementation (`flash_attention_kernel.cu`); online-softmax recurrence; causal-mask handling; `ctypes`/`nvcc` build integration into `combine.so`.
- **Divyansh Sood:** MiniTorch autograd integration (`Tensor.flash_attention`, `FlashAttentionFn`); recomputation-based backward; correctness/parity test suite; benchmark harness (`benchmark_flash_attenti` end-to-end report generation (`presentation_report.json`).

- **Yuvraj Ahuja:** Microbenchmark and memory-profiling design; sequence-length scaling sweep; IWSLT14 BLEU pipeline and matched ablation runs; data analysis and figure generation; final report and poster.

Code and Reproducibility

The full implementation, test suite, benchmark scripts, and the JSON report used to generate the tables and figures in this paper are available at: `<TBD:projectrepositoryURL>`.

References

[1] I. Beltagy, M. E. Peters, A. Cohan. "Longformer: The Long-Document Transformer." arXiv:2004.05150, 2020.

[2] R. Child, S. Gray, A. Radford, I. Sutskever. "Generating Long Sequences with Sparse Transformers." arXiv:1904.10509, 2019.

[3] K. Choromanski et al. "Rethinking Attention with Performers." *ICLR*, 2021.

[4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *NeurIPS*, 2022.

[5] T. Dao. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." *ICLR*, 2024.

[6] B. Lefaudeux et al. "xFormers: A Modular and Hackable Transformer Modelling Library." https://github.com/facebookresearch/xformers, 2022.

[7] M. Milakov, N. Gimelshein. "Online Normalizer Calculation for Softmax." arXiv:1805.02867, 2018.

[8] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, T. Dao. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision." arXiv:2407.08691, 2024.

[9] P. Tillet, H.-T. Kung, D. Cox. "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations." *MAPL*, 2019.

[10] A. Vaswani et al. "Attention Is All You Need." *NeurIPS*, 2017.

[11] S. Wang, B. Li, M. Khabsa, H. Fang, H. Ma. "Linformer: Self-Attention with Linear Complexity." arXiv:2006.04768, 2020.